

动物园

求num数组：对于字符串S的前i个字符构成的子串，既是他的后缀同时又是他的前缀，并且还后缀和前缀不重叠，讲这种字符串的数量记做num[i]，例如S=aaaaa，则num[4]=2（a和aa），aaa虽然满足前后缀相同，但是不满足不重叠，所以不算。先要求一个字符串中num[i]+1之积。

输入：

3

aaaaa

ab

abcababc

输出：

36

1

32

动物园

该题题目中说的很明显，肯定是用kmp求解，很容易想到暴力方法，对于每个位置求next数组的时候，再一直往next的位置跳，判断 $\text{next} \times 2$ 和当前长度的大小，如果满足就条件，就累加，这样就可以直接求出来每个位置num的值，时间复杂度为 $n \times n$ 。很明显过不了。

看数据范围1000000，那么很明显就是 $O(n)$ 的算法，也就是类似于递推，考虑 $\text{num}[i]$ 和 $\text{num}[i-1]$ 的关系或者 $\text{num}[i]$ 和 $\text{num}[\text{next}[i]]$ 的关系。

在寻找关系的时候我们会发现，如果要考虑是否重叠，关系很复杂，一会+1一会不加，单如果先不考虑重叠，让 $\text{num}[i]$ 直接表示公共前后缀的数量，那么关系就很清晰明了：

$$\text{num}[i] = \text{num}[\text{next}[i] - 1] + 1$$

为了方便，这里的前后缀都包含本身，所以 $\text{num}[0] = 1$

动物园

接下来就是要去除掉重叠部分了，对于位置 i ， $\text{next}[i]=j$ ，如果 $j*2>i+1$ ，那么说明当前的 j 是不满足条件的，所以我们直接让 $j=\text{next}[j-1]$ 。如果满足条件，我们以前求的 $\text{num}[j]$ 的值就是我们需要的答案(包含自身的公共前后缀的数量)，有人可能会觉得每次查找的时候 $j=\text{next}[j-1]$ ，万一数据很坑呢？实际上不影响，对于每次 i 来说， $j\leq i/2$ ，那么对于下一个位置 $i+1$ ， j 的变化只有两种情况 $j+1$ 或者 $j=\text{next}[j-1]$ ，前面 j 最多增加一位，那么就算 $j>i/2$ ，那么最多一次 $j=\text{next}[j-1]$ 就可以保证 $j\leq i/2$ ，所以 `while` 完全可以写成 `if`，不影响结果。

动物园

```
for(int i=1;i<l;i++)
{
    while(s[i]!=s[j]&& j)//对于上一次的最大不重复匹配长度为j
    //下一个的最大匹配只有两种情况，第一种是j++，不可能更大了
    //还有一种是s[i]!=s[j]，说明还需要到前面去找，怎么找到，用nextn数组。
    {
        j=nextn[j-1]; //在前面找最长公共前后缀，原理和next数组一样
    }
    if(s[i]==s[j])
    j++;
    while(2*j>i+1)//判断当前匹配长度是否过了一半，这里的while实际上在后面的操作中就是if的作用
    //因为我每次都保证了j<=一半，下一次最多j++超过一半，只需要一次next就小于一半，所以不会超时
    {
        j=nextn[j-1];
    }
    if(j!=0)
    ans=ans*(num[j-1]+1)%1000000007;
}
```

REBXOR

给定一个含 N 个元素的数组 A ，下标从 1 开始。请找出下面式子的最大值：

$$(A[l_1] \oplus A[l_1 + 1] \oplus \cdots \oplus A[r_1]) + (A[l_2] \oplus A[l_2 + 1] \oplus \cdots \oplus A[r_2])$$

其中 $1 \leq l_1 \leq r_1 < l_2 \leq r_2 \leq N$ 。式中 $x \oplus y$ 表示 x 和 y 的按位异或

输入：

5

1 2 3 1 2

输出：

6

$$n < 4 * 10^5 \quad a_i \leq 2^{32}$$

REBXOR

我们先思考如果只选择一个区间应该怎么做，也就是选择一个区间 $[l, r]$ ，让区间 $[l, r]$ 的所有数的异或值最大。

直接做很麻烦，我们可以把区间变成前缀，我们知道相同的两个数异或值为0，那么区间 $[l, r]$ 的异或，我们可以转换成两个前缀 $[1, l-1]$ 和 $[1, r]$ 的异或结果，因为都包含了 $[1, l-1]$ ，所以前面部分异或结果为0。

那么转换成前缀又该怎么做呢？ l 和 r 应该如何选择呢？我们前面学习的字典树只能做选择两个数的最大异或。那么我们就把前缀异或值看做一个数。即 $\text{xor}[i]$ 表示第1个数到第 i 个数的异或值，那么我们只需要找到两个数 $\text{xor}[i]$ ， $\text{xor}[j]$ 的异或值最大就可以了

这样就转换成一道字典树的裸题了，我们把 $\text{xor}[i]$ 的答案都放入字典树中。

REBXOR

我们先思考如果只选择一个区间应该怎么做，也就是选择一个区间 $[l, r]$ ，让区间 $[l, r]$ 的所有数的异或值最大。

直接做很麻烦，我们可以把区间变成前缀，我们知道相同的两个数异或值为0，那么区间 $[l, r]$ 的异或，我们可以转换成两个前缀 $[1, l-1]$ 和 $[1, r]$ 的异或结果，因为都包含了 $[1, l-1]$ ，所以前面部分异或结果为0。

那么转换成前缀又该怎么做呢？ l 和 r 应该如何选择呢？我们前面学习的字典树只能做选择两个数的最大异或。那么我们就把前缀异或值看做一个数。即 $\text{xor}[i]$ 表示第1个数到第 i 个数的异或值，那么我们只需要找到两个数 $\text{xor}[i]$ ， $\text{xor}[j]$ 的异或值最大就可以了

这样就转换成一道字典树的裸题了，我们把 $\text{xor}[i]$ 的答案都放入字典树中。

REBXOR

第一个问题解决了，但是还有一个问题，不是求一个区间最大，而是两个区间异或和之和最大，所以我们还不能先选一个最大的，再重剩下的里面再选一个最大的。

我们这个时候就可以考虑动态规划的思想了，我们可以开一个数组 $l[i]$ 表示从区间 $[1, i]$ 之间选择两个数的最大异或值。依次把每一个前缀加进来，每加入一个数，就求解新加入这个数和前面这些数的一个最大异或值，在和前一个的 $l[i-1]$ 取一个最大值，当做 $l[i]$ 的值。

那么对于后面的部分应该如何选择区间呢？其实我们可以看做后缀和，求解方法和前面求解 $l[i]$ 的方法一样，记做 $r[i]$ 。

我们再枚举 i ，把 $l[i]$ 和 $r[i]$ 相加，注意不能重叠

KMP和字典树

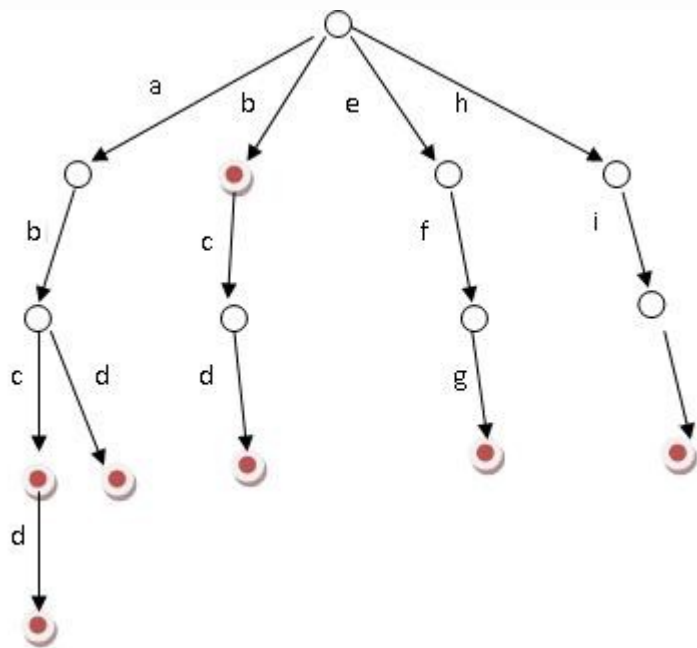
KMP算法主要是用来求解单串匹配的，每次单串匹配的时间复杂度为 $O(n)$ ，但是用于多串匹配他的时间复杂度仍然很慢。

字典树算法主要用于多串匹配，但是他的使用是有限制的，主要是用来求解字符串判等，以及字符串前缀判等的，并不能用来判断一个字符串是否是很多字符串的子串。

那么如果是求解一个字符串是否是多个字符串的子串，那么我们又该如何求解呢？

KMP和字典树

我们首先思考一下，如果我一定要在字典树上做子串的判断，那么我们应该怎么在字典树上实现呢？



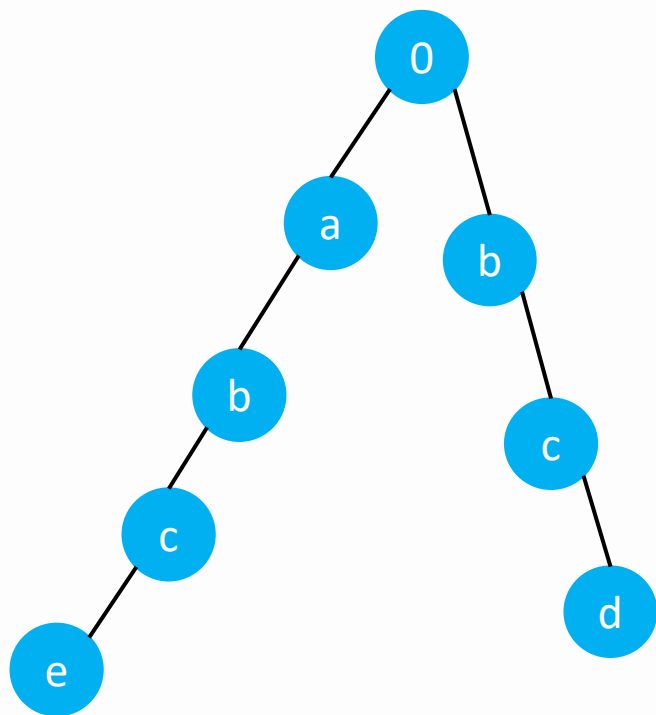
其实很简单，从字符串的开头开始匹配，如果刚好匹配完一条完整的链，那么说明存在子串，但是如果匹配在中途就失败呢？因为字典树是前缀，所以我们又得退回字符串第二个字符，重头开始匹配。实际上就是暴力匹配。

KMP和字典树

我们再学习KMP算法的时候都知道，暴力匹配算法是可以用KMP算法来优化的，那么在字典树上，我们是否也可以用kmp的方式，通过寻找公共前后缀来实现在字典树上直接跳转呢？

也就是说，对于字典树一条链的每一个位置，是否可以在其他链上找到一个公共前后缀(该位置的后缀和另一条链的前缀相等)

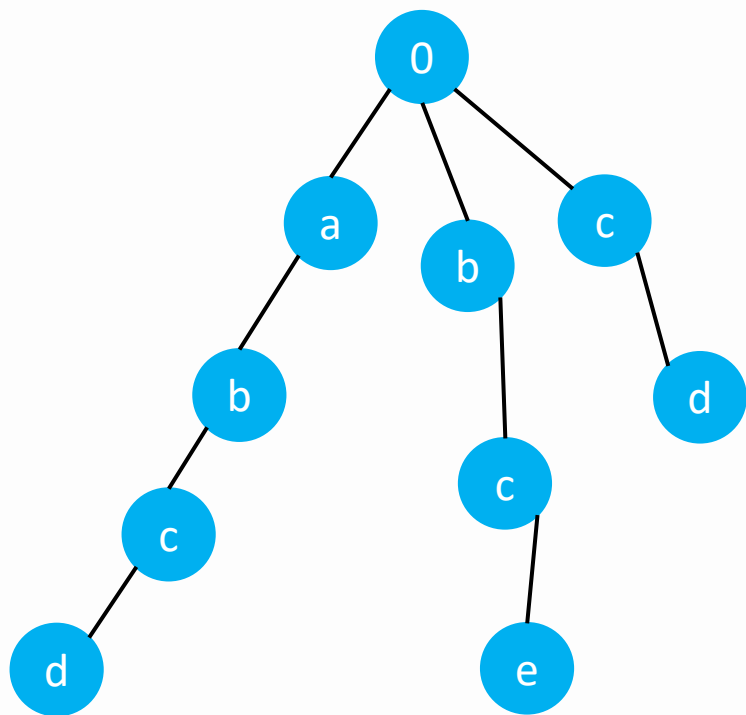
字典树



如果输入两个字符串abce和bcd，判断是否存在字符串abcd的子串。

我们先把abcd从头开始匹配，匹配到c的时候就匹配失败了，但是c并不是单词的末尾，那么表示这条链匹配失败，但是我们会发现对于已经匹配的部分abc，是存在一个后缀bc和另一个单词(bcd)的前缀(bc)相同，既然abc都可以匹配，那么他的后缀也可以匹配的，那么这个前缀也就自然可以匹配，那么我们是不是就可以接着bcd的c的后一个字符开始匹配了。

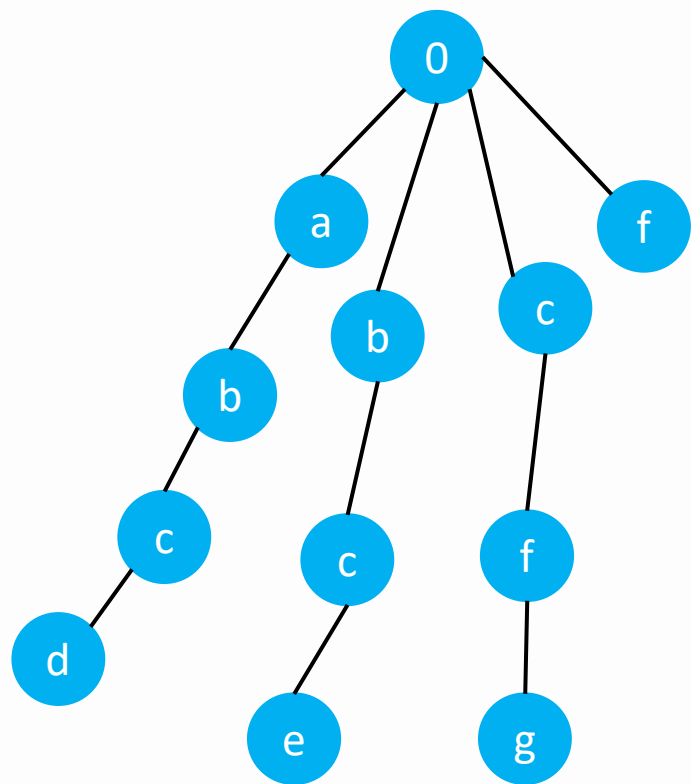
字典树



给定字符串`abcf`, 查找字典树中是否存在他的子串。

首先第一次匹配到`abc`, 匹配失败, 然后发现有一个公共前后缀`bc`, 那么接着匹配, 发现`d!=f`, 匹配失败, 此时已经匹配了`bc`, 又有一个字符串`cfg`的前缀`c`和已经匹配过的字符串的后缀相同, 那么跳转到`cfg`这个字符串, 然后匹配`f`, 发现可以匹配, 但是`f`并不是单词结尾, 匹配失败, 继续跳到相同前后缀`f`去, `f`是单词结尾, 匹配成功。

字典树



给定字符串`abcf`, 查找字典树中是否存在他的子串。

首先第一次匹配到`abc`, 匹配失败, 然后发现有一个公共前后缀`bc`, 那么接着匹配, 发现`d!=f`, 匹配失败, 此时已经匹配了`bc`, 又有一个字符串`cfd`的前缀`c`和已经匹配过的字符串的后缀相同, 那么跳转到`cfd`这个字符串, 然后匹配`f`, 发现可以匹配, 但是`f`并不是单词结尾, 匹配失败, 继续跳到相同前后缀`f`去, `f`是单词结尾, 匹配成功。

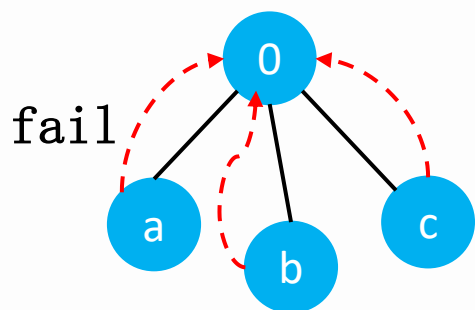
AC自动机

我们会发现，在字典树上寻找子串的过程其实就是kmp算法不断寻找公共前后缀的过程，这样查找子串就不会出现重复，时间复杂度就比较低，而这种把KMP算法和字典树结合起来的方法就是AC自动机。

接下来我们主要是思考如何来记录每一个位置的next数组的值。

因为字典树中已经有next数组了，所以我们定义这个指针为fail指针，也就是失配指针，也就是说如果匹配失败，应该跳到哪一个位置去继续接着匹配。

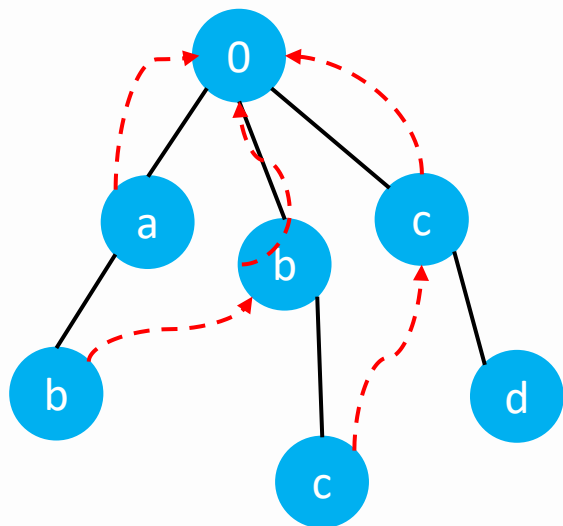
AC自动机



对于0号节点（根节点），他本来就是一个虚拟节点，所以他的失配指针就是指向他本身，或者0号节点不需要失配指针。

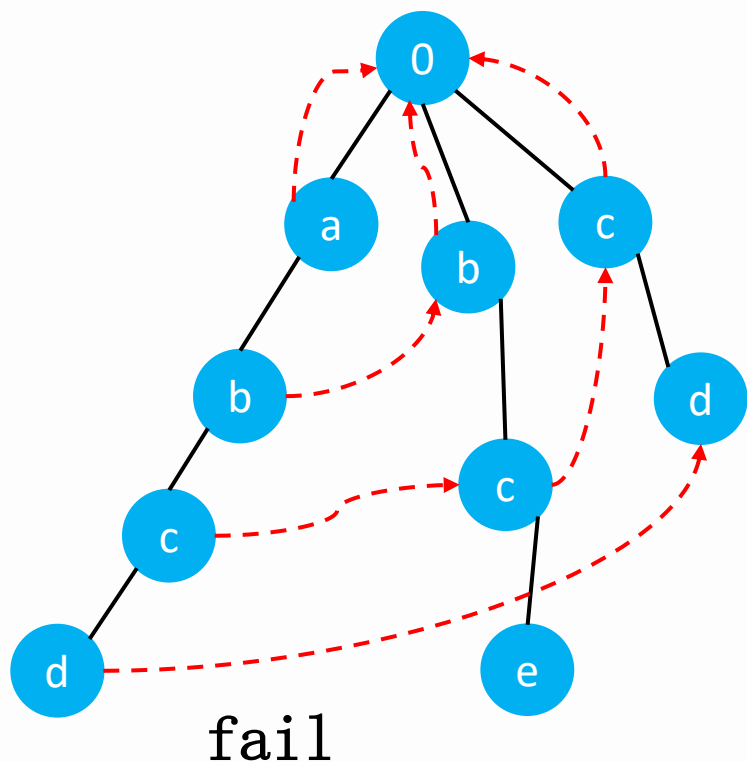
对于第一层的节点，因为公共前后缀不能包括它本身，所以只有一个字符，所以第一层的节点的最长公共前后缀就是0，那么我们就可以让他指向根节点。

字典树



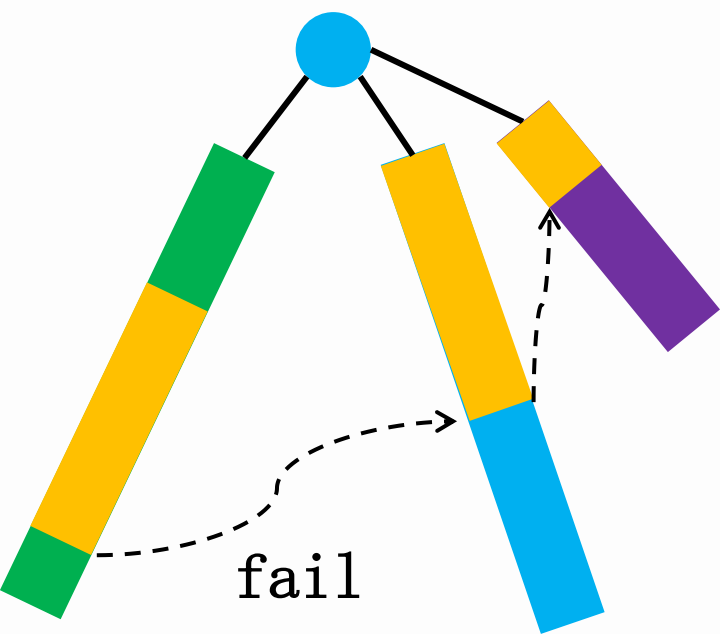
对于第二层的节点，我们设该字符为 x ，那个该节点是上一个节点的第 $x - 'a'$ 个儿子，对于上一个节点，他的失配指针的意义就是有一个后缀=前缀，也就是上一个节点的字符=失配指针指向的字符。那么我们就比较第二层的节点和上一层节点的失配指针的第 $x - 'a'$ 个节点是否存在，如果存在，表示也有这个字符，那么最长公共前后缀+1，所以直接让失配指针直线该位置。如果这个节点不存在，那么说明匹配失败。

字典树



对于后面的节点，失配指针就和第二层是一样的，先找到他上一个节点的失配指针指向的位置，再比较该字符和上一个字符的fail指针的下一个位置，如果可以匹配，那么就把该字符的fail指针指过去，如果不可以匹配呢？是否是直接指向0号节点呢？大家参考一下kmp算法的求法，如果跳到最长公共前后缀之后没有匹配到，并不是直接说不存在，而是应该继续去找公共前后缀，直到匹配到或者没有公共前后缀为止

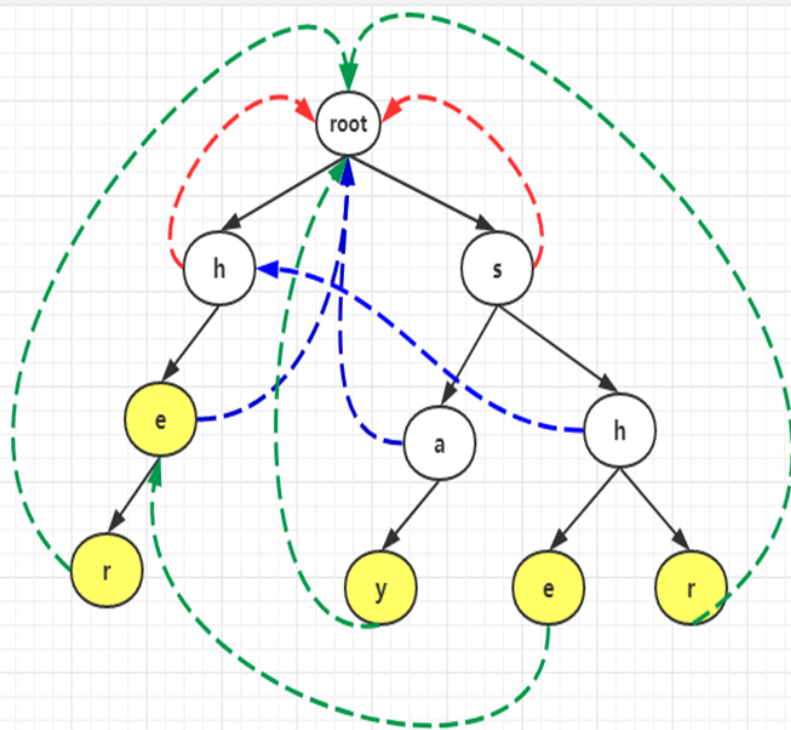
字典树



失配指针的作用在于，当匹配到左边字符串的黄色部分的时候，如果发现不可以继续匹配，只是代表该链的字符串不是给定字符串的子串，那么就需要继续匹配，和kmp一样，没必要换到下一个字符串开头继续比较，因为通过失配指针，我们可以发现右边字符串的黄色部分已经被匹配过了，可以接着匹配。同样，如果该字符下面没有这个字符了，应该跳到fail指针指向的位置的下一个位置去。所以我们还可以修改一下next数组。

算法流程

she
he
say
shr
her
yasherhs



算法流程

如图所示，首先root最初会进队，然后root, 出队，我们把root的孩子的失败指针都指向root。因此图中h, s的失败指针都指向root, 如红色线条所示，同时h, s进队。

接下来该h出队，我们就找h的孩子的fail指针，首先我们发现h这个节点其fail指针指向root, 而root又没有字符为e的孩子，则e的fail指针是空的，如果为空，则也要指向root, 如图中蓝色线所示。并且e进队，此时s要出队，我们再找s的孩子a, h的fail指针，我们发现s的fail指针指向root, 而root没有字符为a的孩子，故a的fail指针指向root, a入队，然后找h的fail指针，同样的先看s的fail指针是root, 发现root又字符为h的孩子，所以h的fail指针就指向了第二层的h节点。e, a, h 的fail指针的指向如图蓝色线所示。

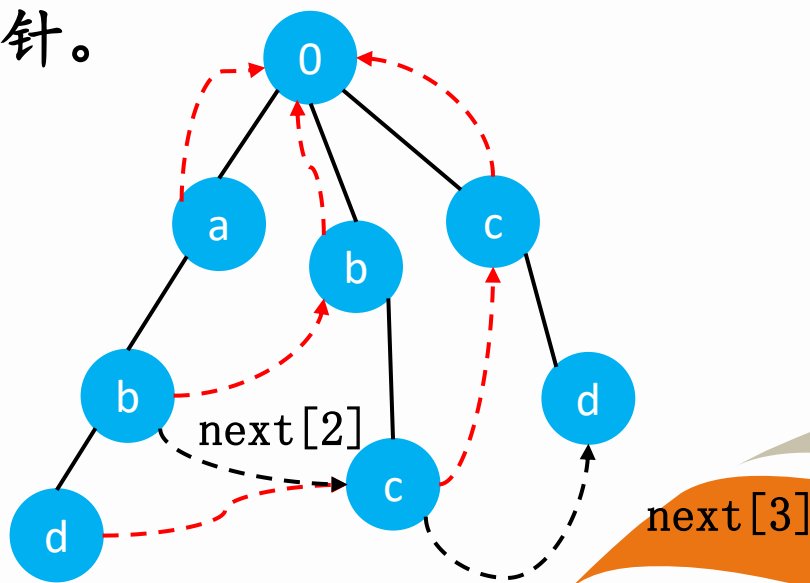
此时队列中有e, a, h, e先出队，找e的孩子r的失败指针，我们先看e的失败指针，发现找到了root, root没有字符为r的孩子，则r的失败指针指向了root, 并且r进队，然后a出队，我们也是先看a的失败指针，发现是root, 则y的fail指针就会指向root. 并且y进队。然后h出队，考虑h的孩子e, 则我们看h的失败指针，指向第二层的h节点，看这个节点发现有字符值为e的节点，最后一行的节点e的失败指针就指向第三层的e。最后找r的指针，同样看第二层的h节点，其孩子节点不含有字符r，则会继续往前找h的失败指针找到了根，根下面的孩子节点也不存在有字符r，则最后r就指向根节点，最后一行节点的fail指针如绿色虚线所示。

AC自动机

```
void fail_point(int id){
    while(!q.empty()) q.pop();
    for(int i=0;i<26;i++){
        int j=tree[id].next[i];
        if(j!=0)//把第一层结点放入队列中，即源点直接连接的点
        {
            tree[j].fail=id;//第一层指针的失配指针都是根结点
            //因为第一层结点肯定都是各不相同的，所以第一层结点只能指向他本身。
            q.push(j);//我们依次把一层一层的结点都放进去。
        }
    }
    while(!q.empty()){
        int now=q.front(); q.pop();
        for(int i=0;i<26;i++){
            int j=tree[now].next[i];
            if(j==0)//now下面的当前位置没有结点了
            {
                tree[now].next[i]=tree[tree[now].fail].next[i];
                continue;
                //如果当前结点下方没有结点了，那么为了避免重复，我们应该找到他的子串的接下来的位置继续匹配
                //比如 abcd bcde cdef
                //当abcd匹配完之后，实际上bcd也匹配完了。我们应该接着bcd之后去匹配e。
                //同样的，当e匹配完之后，我们应该接着cde之后匹配f
                //我们会发现前一个串的后缀等于后一个串的前缀的时候我们才可以匹配，否则不可以
            }
            tree[j].fail=tree[tree[now].fail].next[i]; q.push(j);
        }
    }
}
```

AC自动机

这里除了给每一个位置添加了fail指针，还给单词的末尾添加了一个next指针。因为我们字典树遍历的时候都是一直去找当前节点的next节点，那么如果在当前这条链上匹配失败，那么应该跳到最后一个匹配成功的节点的fail指针的下一个位置，那么我们这里为了方便，可以直接让最后一个节点的next指针指向该位置，这样我们就可以一直通过next指针遍历下去，而不是一会遍历next指针，一会遍历fail指针。



匹配过程

当AC自动机搭建好了之后，接下来就是匹配了，对于一个给定字符串，比如要求多少个文本串在其中出现过，那我们就可以先匹配，因为比如有abcd, bcd, cd, 当abcd匹配完之后实际上bcd, cd都匹配完了，所以在匹配过程中，找到一个字符的时候，应该继续去找他的失配指针以及失配指针的失配指针，判断是否是单词的结尾，当找完之后，再回到最开始的那条链继续往下面找。

因为在我们搭建AC自动机的时候，因为把每个字符串的最后一个字符的下一个位置链接到其失配指针的下一个位置了，所以只需要一直遍历next就可以了。

源代码

```
void search(int id ,char *s)
{
    int l=strlen(s);
    for(int i=0;i<l;i++)
    {
        int j=tree[id].next[s[i]-'a'];
        while(j!=0&&tree[j].flag!=-1)
            //flag即表示是单词，也表示了单词的个数
            //如果该单词匹配过了，再下次仍然可以匹配的时候就不在匹配了
            //否则一个单词会重复计算。
            //比如ab ababab,ab只应该算一次
        {
            ans+=tree[j].flag;//对于非单词结尾，都是0，对结果不影响
            tree[j].flag=-1;//算过了就不再算了，标记为-1.
            j=tree[j].fail;//找到还有没有这个字符
            //比如当前是***a,他会继续查找其他链里面是否还有前缀a,
            //相当于一层一层的找
        }
        id=tree[id].next[s[i]-'a'];//相当于第一条链为主链，依次查找是否有相同
        //不能写成j，因为j改变了
    }
}
```

单词

给定一篇文章的所有组成单词，问文章中的每一个单词在文章中出现了多少次。

输入：

3

a

aa

aaa

输出：

6

3

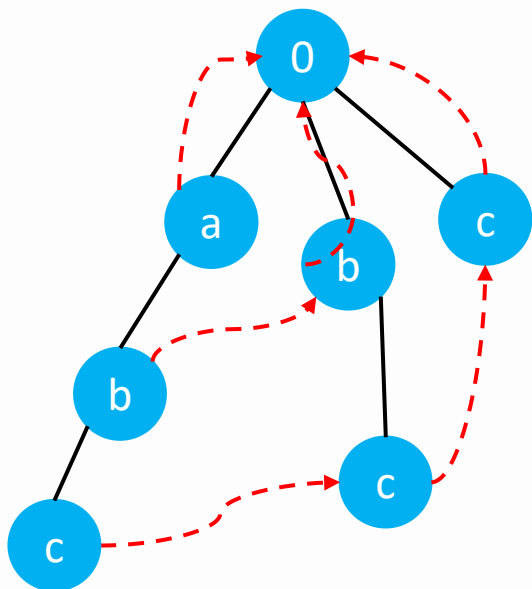
1

原文为 a aa aaa, 那么单词a出现了6次，单词aa出现了三次(在aaa中算出现了两次)，aaa出现了一次。

单词

该题并不是问这个字符串是否出现过，而是问这个字符串是否是其中哪些字符串的子串(只要该字符串被包含在内都算)。而且该题是反过来的，不是问你字典树上哪些字符串是该字符串的子串，而是问该字符串是字典树上哪些字符串的子串。

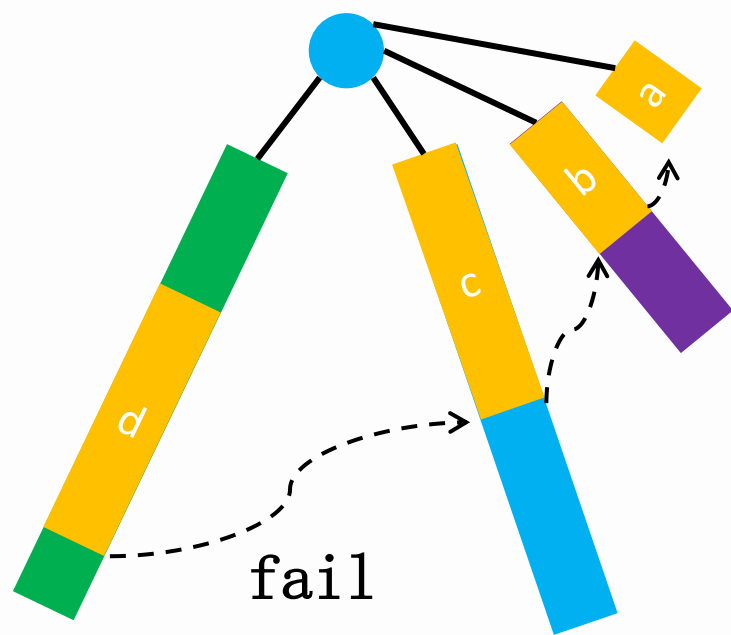
假设有三个单词abc, dc, c



我们要查找单词c出现的次数，应该是3次，但是如果我们去AC自动机中去查找c，只能找到一个。

那我们应该如何处理呢？

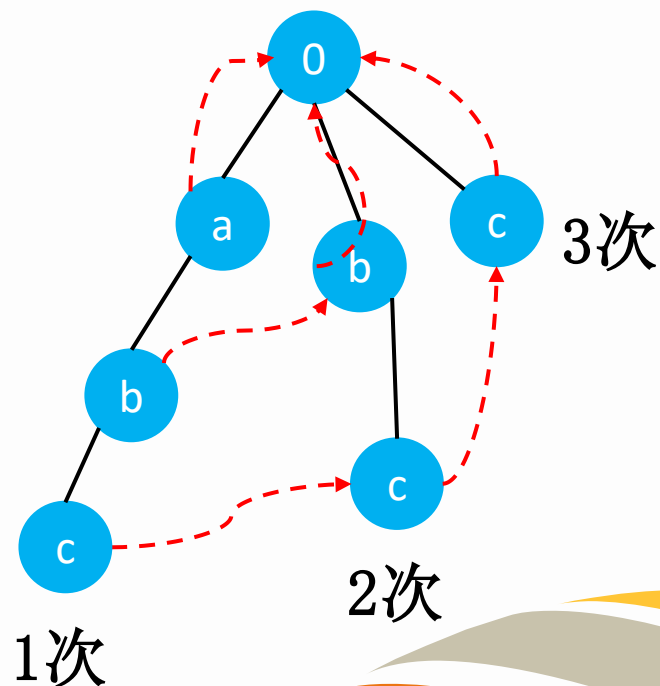
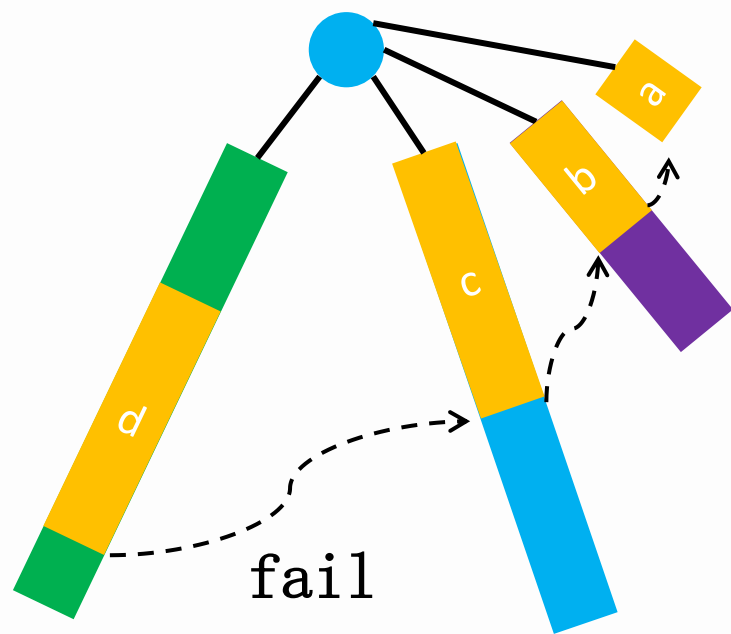
单词



假设我们要查找黄色部分a这个单词出现的次数，那么如果有一个单词的fail指针指向了a这个单词的末尾，那么就说明a单词在b单词中出现过。出现次数+1，我们继续往下面看，如果有一个单词c的fail指针指向了b，那么说明单词b存在于单词c中，所以单词a也存在单词c中，出现次数+1. 我们可以一直这样找下去，直到找不到为止，中途没fail指针跳一次，那么出现次数就会+1次。

单词

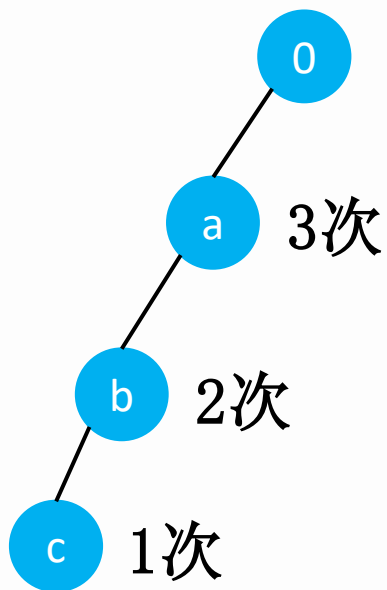
那么我们就可以开一个sum数组，表示有多少个包含的单词，那么 $\text{sum}[\text{fail}[i]] += \text{sum}[i]$



单词

第一个问题解决了，但是这个sum的值该如何赋值呢？
每添加一个节点，那么sum就+1.

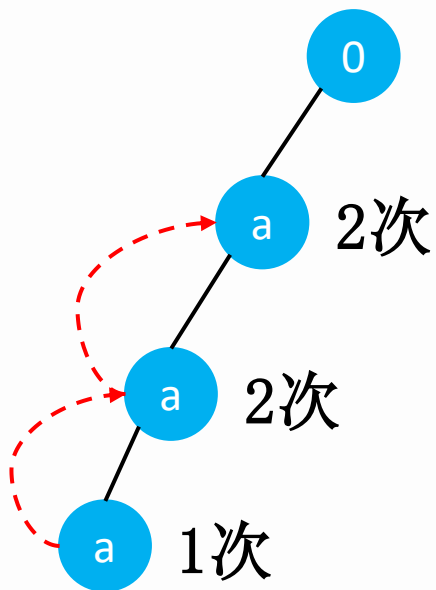
比如插入abc, ab, a。



单词

但是这样做有一个很大的问题，也就是说如果一个单词中如果一个字符或者一截字符串重复了多次，这样统计答案就错了。

比如aaa,, 单词a出现的次数应该是3次，但是这里明显只有2次。 $\text{sum}[1]=\text{sum}[2]+\text{sum}[1]=2$
 $\text{sum}[2]=\text{sum}[3]+\text{sum}[2]=2$



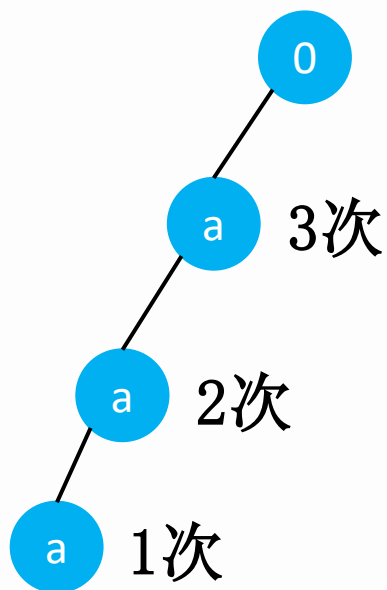
所以我们统计sum数组的时候应该在fail指针全部建完之后再单独统计，不能边建边统计。

同时统计的时候应该从链的底端开始往上面统计。

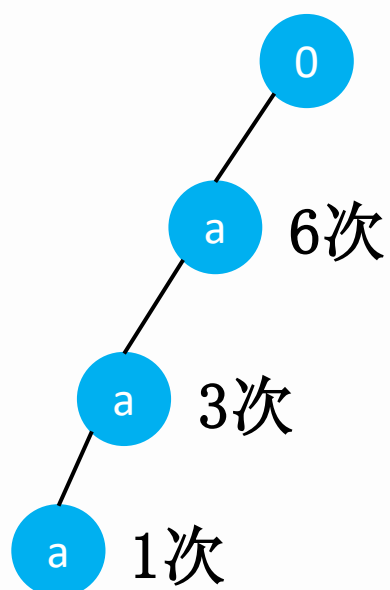
单词

比如aaa, aa, a, 我们来尝试一下

先全部插入



再统计





AC自动机练习题

洛谷 3808 AC自动机
洛谷 3796 AC自动机
洛谷 5357 AC自动机
BZOJ 3172 单词
HDU 2222 Keywords
BZOJ 4327 玄武密码
BZOJ 3940 Censoring
BZOJ 1195 最短母串
BZOJ 2938 病毒
BZOJ 1030 文本生成器
洛谷 2336 喵星球上的点名

